Distributed OCR System

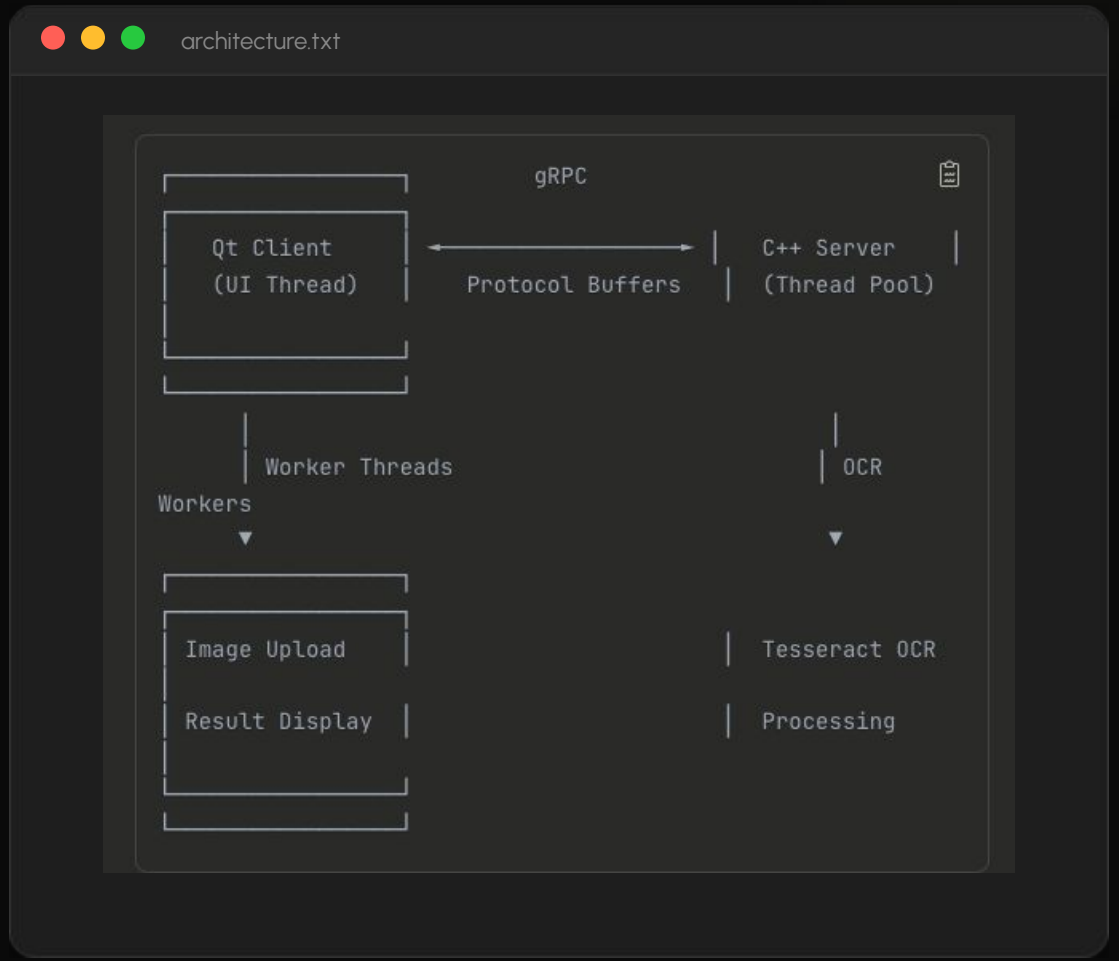
A Robust Client-Server Architecture for Optical Character Recognition

STDISCM Problem Set 4

By Gavrie Clemente & Robee Feliciano

System Architecture

- **Qt Client (UI Thread):** Handles asynchronous image uploads and result display.
- **gRPC & Protobuf:** Ensures efficient binary communication.
- **C++ Server:** Manages concurrent OCR tasks using a thread pool.



Server-Side Multithreading

OCRThreadPool.cpp

```
class OCRThreadPool { std::vector<std::thread> workers;
std::queue<Task> tasks;
std::vector<std::unique_ptr<TessBaseAPI>> engines; }; void
worker_thread(size_t id) { while (!stop) { Task task; {
std::unique_lock lock(queue_mutex); cv.wait(lock, [this]{
return stop || !tasks.empty(); }); task =
std::move(tasks.front()); } // Process safely in own thread
process_ocr(task); } }
```

Thread Pool Architecture

The server prevents resource exhaustion by using a fixed pool of workers.

- **Worker Scaling:** `std::thread::hardware_concurrency()` determines the thread count.
- **Task Queue:** Decouples request acceptance from processing.
- **Engine Isolation:** Each thread has a unique `TessBaseAPI` instance.

Client-Side Concurrency

Non-Blocking UI

The Qt client uses `QThread` to keep the UI responsive during network operations.

- **Thread Creation:** Worker thread created for each upload.
- **Async Signal:** `Q_EMIT` updates the UI safely upon completion.
- **Cleanup:** `deleteLater` ensures no memory leaks.

OcrClient.cpp

```
void OcrClient::processImage(const QString &path) { // Create worker thread to avoid blocking UI QThread *thread = QThread::create([this, path]() { ocr::ImageRequest request; // ... read file & setup request ... Status status = stub_->ProcessImage(&ctx, request, &res); // Update UI safely via signal Q_EMIT imageProcessed(path, res.text(), res.success()); }); connect(thread, &QThread::finished, thread, &QThread::deleteLater); thread->start(); }
```

Synchronization Primitives

Mutex Protection

std::mutex ensures exclusive access to the shared task queue and OCR engines, preventing data races during concurrent access.

Condition Variables

std::condition_variable efficiently puts worker threads to sleep when the queue is empty and wakes them only when new tasks arrive.

Atomic Counters

std::atomic provides lock-free thread-safe counters for tracking active threads and pending requests for monitoring.

Promise/Future Pattern

The Promise

When a gRPC request arrives, the handler creates a `std::promise` and enqueues the task. This acts as the "producer" side of the result.

The Future

The handler immediately receives a `std::future`. It waits on this future (`future.get()`), effectively blocking only the specific RPC handler thread until the worker thread computes the OCR result.

Qt GUI Architecture

Connection Manager

Manages gRPC channel state, providing visual feedback on connection status and handling retries.

Async Progress

A dynamic progress bar tracks the total vs. processed images, updating in real-time via Qt signals.

Result Grid

A flexible `QGridLayout` displays image thumbnails alongside extracted text, enabling easy verification of OCR output.

gRPC & Protocol Buffers

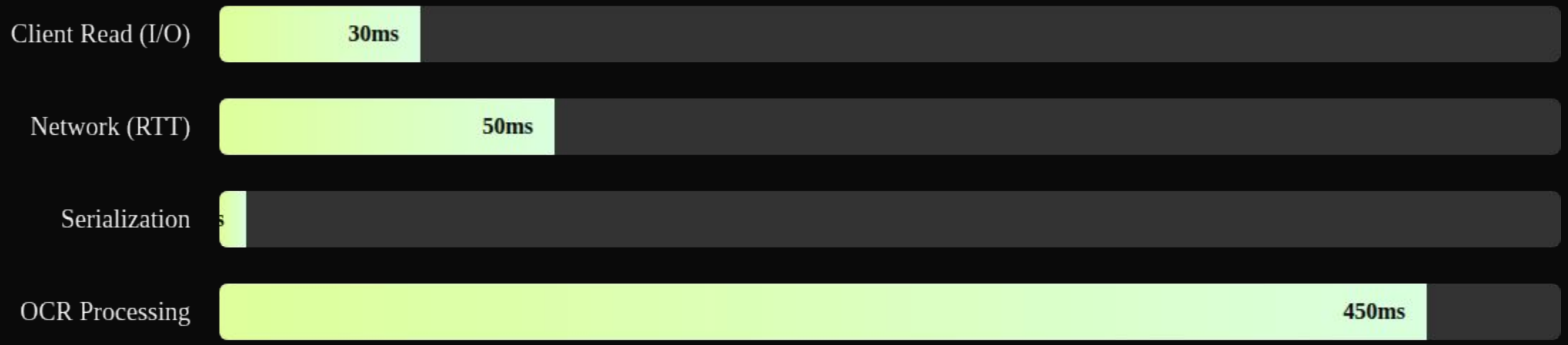
High-Performance IPC

- **Service Definition:** Defined in `.proto` files, ensuring a strict contract.
- **Efficient:** Binary serialization is smaller and faster than JSON.
- **Large Payloads:** Supports large image transfers via `bytes` type.

ocr.proto

```
service OCRService { rpc ProcessImage(ImageRequest) returns (OCRResponse) {} } message ImageRequest { string image_id = 1; bytes image_data = 2; // Raw image bytes string client_id = 3; } message OCRResponse { string image_id = 1; string extracted_text = 2; bool success = 3; string error_message = 4; }
```

Latency Breakdown (Single Request)



OCR processing dominates the total latency, justifying the multi-threaded architecture to handle throughput.

Server Resilience Strategies

OCRServiceImpl.cpp

```
try { // Write temp file & Load image Pix* image =
pixRead(temp_file.c_str()); if (!image) throw
runtime_error("Load failed"); // Perform OCR
ocr_engine->SetImage(image); char* text =
ocr_engine->GetUTF8Text();
response.set_extracted_text(text);
response.set_success(true); pixDestroy(&image); } catch
(const exception& e) { // Catch crash, return error, keep
server alive response.set_success(false);
response.set_error_message(e.what()); }
```

Robust Error Handling

The server implementation wraps critical processing logic in try-catch blocks.

- **Request Isolation:** A crash in one request (e.g., malformed image) does not bring down the server.
- **Graceful Recovery:** Resources are cleaned up, and a structured error response is sent back to the client.

Client-Side Reliability

- **Connection Failures (UNAVAILABLE):** Detected immediately. The UI disables upload controls and prompts the user to check the server address, preventing phantom requests.
- **Request Timeouts (DEADLINE_EXCEEDED):** gRPC deadlines ensure the client doesn't hang indefinitely if the server becomes unresponsive during processing.
- **UI Consistency:** Even if individual image requests fail, the progress bar continues to advance, and error states are clearly marked in the result grid.

Key Architectural Takeaways

Parallelism

Maximizes hardware usage via thread pools.

Performance

Low-overhead gRPC binary communication.

Resilience

Isolated failures and non-blocking UI.

DEMO

Thank you for your attention.

Live Demo Ready